Northeastern University
College of Engineering

IE 7615 — Deep Learning Architectures
Fall 2026

PROJECT 1 — FINAL REPORT

Celebrity Detection and Identification with YOLOv8 and ResNet18

Group 02

Abdellah Faleh

Jin-Woo Hong

Samuel Tong

Tristan Lyons

Submitted: October 21, 2026
Repository: github.com/AHonGa/ie7615-group-02-project1

1. Introduction

The goal of Project 1 is a computer-vision system that takes an image containing several faces, finds each face, and says which celebrity it is. Two tasks are combined: **localization** (draw a box around every face) and **identification** (give each box a name). We built the system in four milestones: a single-face classifier (Milestone 1), a synthetic multi-face detection dataset (Milestone 2), a YOLOv8 detector fine-tuned on that dataset (Milestone 3), and, in this final milestone, an integrated pipeline in which the detector finds faces and the classifier helps name them.

The identities are 13 CelebA celebrities chosen through the class-wide identity pool. Four of them (CelebA ids 3, 7, 1212 and 8335) were claimed by our group; the other nine come from other groups. Each identity has only 23–25 photos, so the central challenge throughout the project has been learning to recognize people from very little data.

Our main results: the detector finds every face in the held-out test grids with almost perfect boxes (mean IoU 0.99); the integrated pipeline names **89 of 108 test faces correctly (82.4%)**, up from 84 (77.8%) for the detector alone, and removes all duplicate boxes. Most remaining errors are confusions between a small group of visually similar identities, which also raises the bias questions discussed in Section 8.

2. Dataset construction

Source data. All images come from the CelebA "aligned and cropped" set (178 × 218 px face photos). For each of the 13 identities we extracted all of its photos (23–25 each, 319 in total) into `extracted_identities/`.

Classification data. In Milestone 1 the classifier used only our 4 identities, split 70/15/15 per identity (67 train / 16 val / 16 test photos). For the integrated system the classifier must know all 13 identities, so in Milestone 4 we reuse the **photo-level split from Milestone 3** (`detection_dataset_m3/photo_split.csv`): each identity's photos were divided 70/15/15 into train, validation and test *before* any grid was built. This gives 219 training, 50 validation and 50 test photos (about 17, 4 and 4 per identity). Because the classifier and the detector share this split, **no test photo is seen by either model during training**, and both models use the same class ids.

Detection data. A detection image is a 640 × 640 canvas holding a 3 × 3 grid of 9 different celebrities. Each face photo is resized to a square that fills 65–92% of its grid cell and placed at a random offset on a plain background, and its YOLO label (class, centre, width, height) is written at the same time, so boxes are exact by construction. The Milestone 2 dataset had 6 base grids with 4 augmented copies each (30 images). In Milestone 3 we found that it used only about 4 of each celebrity's ~25 photos and missed 4 identities in its test split, so the detector could not learn identities (test mAP@0.5 = 0.23). With the TA's approval we rebuilt it with the same method but with the photo-level split above and more grids: **120 train, 12 validation and 12 test grids** (108 test faces), with every identity in every split. All dataset scripts are in `src/` and are deterministic (seed 42).

3. Models and training

Classifier (Milestone 1 architecture). Milestone 1 compared a custom 4-block CNN trained from scratch with an ImageNet-pretrained ResNet18 fine-tuned end to end. On the 4-identity subset the ResNet18 reached 87.5% test accuracy against 50% for the custom CNN, because with ~17 training photos per identity a network learned from scratch cannot form reliable face features, while pretrained features transfer well. We therefore kept the ResNet18 (`src/models.py`) and retrained it for all 13 identities: 128 × 128 input with ImageNet normalization (as in Milestone 1), AdamW (lr $3{\cdot}10^{-4}$, weight decay 10^{-4}), cosine learning-rate schedule, cross-entropy with label smoothing 0.1, batch 32, 30 epochs, seed 42. Training augmentation imitates what detector crops look like: horizontal flips, small shifts and zoom (±5%, ±10%), rotation up to 5°, and brightness/contrast/saturation jitter. The checkpoint with the best validation accuracy is kept (`models/celeba_resnet18_13id.pt`). Training takes about 10 minutes on a laptop CPU.

Detector (Milestone 3). A COCO-pretrained YOLOv8n, fine-tuned for up to 100 epochs (AdamW, lr 0.001, 640 px, batch 16, early-stopping patience 30) with one class per identity. Six configurations were trained (learning rate ×0.2 and ×5, image size 416, lighter online augmentation, and the larger YOLOv8s); the final model, **`aug_light`** (YOLOv8n without mosaic and with gentler colour and scale jitter), was selected on validation mAP only (`models/celeba_yolov8_best.pt`). Table 1 summarizes the training settings of both models.

Setting	ResNet18 classifier	YOLOv8n detector
Pretrained on	ImageNet	COCO
Input size	128 × 128 face crop	640 × 640 grid image
Optimizer, learning rate	AdamW, $3{\cdot}10^{-4}$, cosine schedule	AdamW, lr0 = 0.001
Batch size, epochs	32, 30 (best epoch 5)	16, up to 100 (patience 30)
Loss	Cross-entropy, label smoothing 0.1	YOLOv8 box + class + DFL loss
Augmentation	Flip, ±5% shift, ±10% zoom, 5° rotation, colour jitter	No mosaic, scale 0.2, translate 0.05, light HSV jitter
Training data	219 face photos	120 grids (1,080 faces)
Training time	~10 min on a laptop CPU	Full 6-run sweep ~30–45 min on a Colab T4 GPU

Table 1. Training settings of the two fine-tuned models.

Integrated pipeline (Milestone 4). Figure 1 shows the system (`src/m4_pipeline.py`):

1. **Detect.** YOLOv8 finds faces using *class-agnostic* non-maximum suppression, so overlapping boxes with different names are merged into one box per face. (Ultralytics' default suppresses only within the same class, which is why the Milestone 3 detector sometimes drew two boxes on one face.)
2. **Crop and classify.** Each box is cropped from the image, resized to 128 × 128, and the ResNet18 returns a probability for each of the 13 identities.
3. **Fuse.** The final score for each identity is the average of the classifier's probability and the detector's vote (its confidence for the class it predicted, zero for the others): `score = 0.5 · (p_classifier + vote_detector)`. The name with the highest score wins. When

both models agree, the score is high; when they disagree, the more confident one usually wins.

4. **Fallbacks for unseen images.** The detector was trained on faces that fill one grid cell, so it may find nothing in, for example, a single full-frame portrait. In that case the image is shrunk into one cell-sized patch on a blank canvas and searched again, and as a last resort the whole image is classified as one face. Names with a final score below 0.5 are shown with a "?".

The pipeline runs as one command: `python src/identify.py --source <image or folder>` or, for the live session, `python src/identify.py --app` (a web page where any image can be uploaded). The naming mode (detector only, classifier only, or fusion) was chosen on the validation grids.

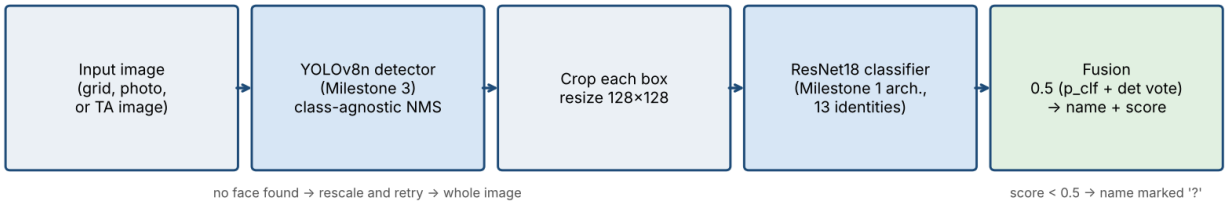


Figure 1. Integrated pipeline.

4. Experimental setup

All numbers below are on held-out data. Model choices (best epoch, detector configuration, naming mode) were made **on the validation split only**; the test split was used once, for reporting.

- **Classifier:** accuracy on the 50 validation and 50 test photos, per identity and as a confusion matrix.
- **Detector:** mAP@0.5, mAP@0.5:0.95, precision and recall from the Ultralytics validator, and mean IoU of correctly identified faces from our own matcher.
- **Pipeline:** each prediction is matched to the unclaimed ground-truth face it overlaps most (IoU ≥ 0.5). A face counts as **correct** only if it is found *and* given the right name; we also count wrong names, missed faces and extra boxes. This "faces correctly identified" figure is our headline metric because it is what a user of the system sees.
- **Robustness:** harder copies of test grids (faces shrunk to 45%, exposure cut to 30%, heavy blur, two grids squeezed into one image), plus one unseen test image supplied by the TA.

5. Results

Detector (Milestone 3, test grids). mAP@0.5 = 0.879, mAP@0.5:0.95 = 0.876, precision 0.849, recall 0.759, mean IoU 0.993. The detector located all 108 test faces; 84 were given the right name. Learning rate was the most influential setting (×5 higher dropped validation mAP@0.5 from 0.90 to 0.74), and lighter online augmentation was best (validation mAP@0.5:0.95 0.947 vs 0.896 for the baseline). On the original 30-image dataset the same configuration reached only 0.232. Table 2 lists the full sweep.

Run	Model	lr0	Image size	Val mAP@0.5	Val mAP@0.5:0.95	Test mAP@0.5
baseline	YOLOv8n	0.001	640	0.897	0.896	0.799
lr_low	YOLOv8n	0.0002	640	0.864	0.863	0.861
lr_high	YOLOv8n	0.005	640	0.737	0.737	0.687
imgsz_416	YOLOv8n	0.001	416	0.835	0.835	0.847
aug_light (selected)	YOLOv8n	0.001	640	**0.947**	**0.947**	**0.879**
yolov8s_aug_light	YOLOv8s	0.001	640	0.933	0.933	0.936

Table 2. Detector hyperparameter sweep (Milestone 3). The run was selected on validation mAP.

Figure 2 shows its training curves: box loss converges for training and validation alike, while the validation classification loss stays well above training, the first sign that naming, not finding, is the hard part.

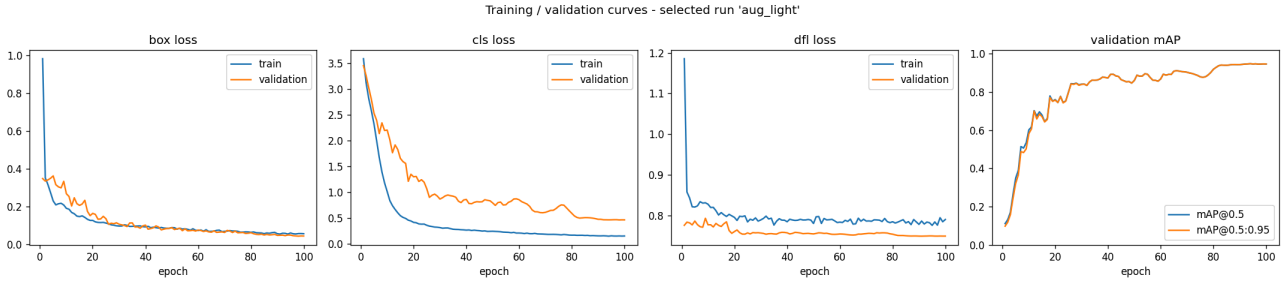


Figure 2. Detector training and validation curves (selected run).

Classifier (single face photos). The 13-identity ResNet18 reached **84% validation accuracy (42/50)** and **78% test accuracy (39/50)**. Training accuracy reached 100% by epoch 7 while validation accuracy levelled off around 80% (Figure 3, left), so the model overfits its ~17 photos per identity; the best validation checkpoint (epoch 5) is used. Seven of the 13 identities were classified perfectly on test, including three of our four (celeb_3, celeb_7, celeb_8335); the weakest were celeb_4422 (1/4), celeb_4428 (1/3), and celeb_1212 and celeb_7007 (2/4 each). Table 3 gives the full breakdown.

Identity	Test photos	Correct	Accuracy		Identity	Test photos	Correct	Accuracy
celeb_3 (ours)	4	4	100%		celeb_4422	4	1	25%
celeb_7 (ours)	4	4	100%		celeb_4428	3	1	33%
celeb_797	4	4	100%		celeb_5695	4	3	75%
celeb_1212 (ours)	4	2	50%		celeb_7007	4	2	50%
celeb_2336	4	4	100%		celeb_8335 (ours)	4	4	100%
celeb_2619	4	4	100%		celeb_10002	3	3	100%
celeb_2970	4	3	75%		**All 13**	**50**	**39**	**78%**

Table 3. Classifier test accuracy per identity (single face photos).

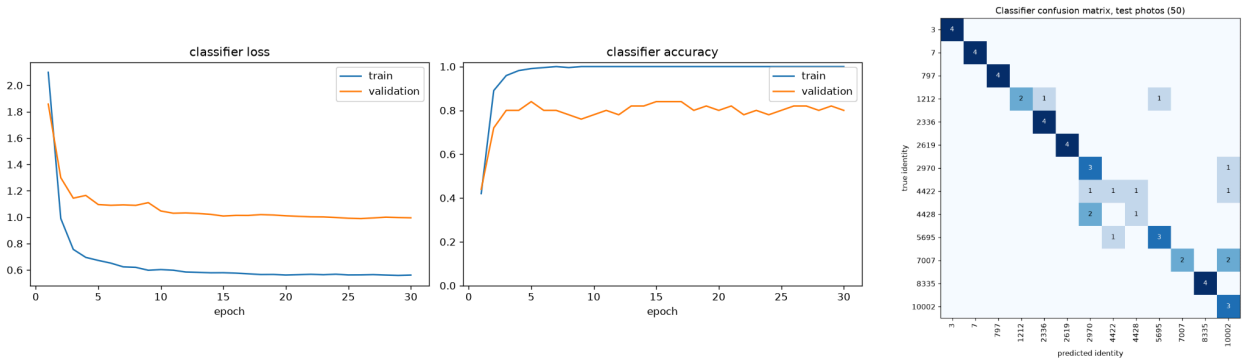


Figure 3. Classifier learning curves (left) and test confusion matrix (right).

Integrated pipeline (multi-face grids). Table 4 compares the four settings. On both validation and test, fusion is best:

Setting	**Val correct (of 108)**	**Test correct (of 108)**	**Test wrong name**	**Test extra boxes**	**Test mean IoU**
Milestone 3 detector alone (per-class NMS)	95 (88.0%)	84 (77.8%)	24	8	0.993
Detector alone, one box per face	95 (88.0%)	84 (77.8%)	24	0	0.993
Detector boxes + classifier names	89 (82.4%)	80 (74.1%)	28	0	0.991
Detector + classifier votes combined (selected)	**98 (90.7%)**	**89 (82.4%)**	**19**	**0**	**0.991**

Table 4. Pipeline settings on validation and test grids. The setting was chosen on validation.

Fusion names 5 more test faces correctly than the detector alone and removes all 8 duplicate boxes; no face was missed. In the gallery, 2 of the 12 test grids are fully correct and every grid has at least 6 of 9 faces right.

Robustness. Table 5 shows the stress copies. Fusion helps most where the detector is weakest (crowded and small faces), but all four conditions remain far below the clean test grids.

Stress test (18 faces each)	**What was changed**	**Milestone 3 detector**	**Fusion pipeline**
Crowded	Two grids squeezed into one image	3 of 18	**6 of 18**
Dark	Exposure cut to 30%	10 of 18	10 of 18
Blurred	Heavy blur	10 of 18	10 of 18
Small faces	Faces shrunk to 45%	1 of 18	**2 of 18**
Clean test grids, for reference	—	*84 of 108 (78%)*	*89 of 108 (82%)*

Table 5. Faces correctly identified on harder copies of test grids.

On the unseen image supplied by the TA (a 3 × 3 grid on a green background, a background never used in training), the detector found all 9 faces and named all 9

correctly with confidence 0.98–1.00.

Final gallery (`results/milestone4/gallery/README.md`, examples in Figure 4): 20 correctly classified single faces (at least one per identity), 11 misclassified single faces with an analysis note and a training photo of the identity they were confused with, and 19 multi-face images (12 test grids and 7 stress tests) with boxes coloured by outcome and captions.



Figure 4. Gallery examples. Top: correctly classified test photos. Bottom: two misclassifications (each with a training photo of the predicted identity), a fully correct test grid, and the crowded stress test.

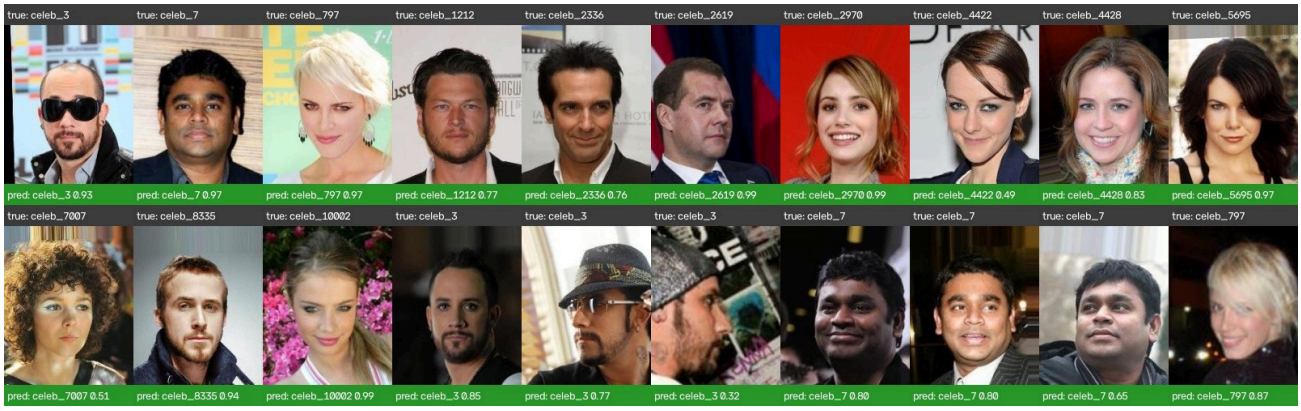


Figure 5. All 20 correctly classified single faces in the gallery (true name on top, prediction and score below), covering every one of the 13 identities.

6. Live demonstration

The final evaluation tests two things on images the team has never seen: **single-object identification** (one face, one name) and **multi-object detection** (every face boxed and named). Both go through the same command, `python src/identify.py`, so nothing changes between the two cases.

Single face. A close-up portrait is the hardest input for our detector, because it was trained on faces that fill one cell of a 640 px grid. The pipeline handles this with the fallbacks described in Section 3: if no face is found, the image is shrunk into a grid-cell-sized patch and searched again, and as a last resort the whole image is classified as one

face. In each case the classifier still names the face, so a single portrait always returns exactly one name and score.

Several faces. For a group image the detector draws one box per face (class-agnostic suppression), every box is cropped and classified, and the fused name is written on the box. Confident names get a green box; a name with a fused score below 0.5 gets an orange box and a "?", so the audience can see when the system is unsure.

What the demo shows. The web app (`python src/identify.py --app`, or `--share` for a public link during the Zoom session) takes an uploaded image and returns the annotated image and a table with one row per face: the final name and score, the detector's vote, the classifier's vote, how the face was found (detector, rescaled image, or whole image), and the box coordinates. Showing both votes makes the fusion step visible: the audience can see the cases where the two models agree and the cases where one model corrects the other. The naming mode (detector, classifier or fusion) and the detector's confidence threshold can be changed on the page, which lets us show the comparison in Table 4 live.

Rehearsal. Before the session we ran the demo on held-out test grids and shared the public link within the team, so the presenter does not need the code installed. In Milestone 3 the detector alone named all 9 faces in the TA's unseen test image correctly. The demo runs on a laptop CPU; no GPU is needed.

7. Analysis and limitations

Finding faces is solved; naming them is the hard part. Across every experiment the detector finds essentially every face with near-exact boxes (IoU ≈ 0.99), so mAP@0.5 and mAP@0.5:0.95 are almost equal. All remaining errors are about *who*, not *where*.

Why fusion helps. The two models make different mistakes. The detector sees each face in the context of the whole grid at 640 px; the classifier sees a tight crop at 128 px and was trained on plain photos. Alone, the classifier is slightly weaker on grids (80 vs 84 correct), but when the two are averaged, a confident correct vote from one model can outweigh an unsure wrong vote from the other, which corrected 5 test faces. Class-agnostic suppression is a separate, free improvement: it removes duplicate boxes without changing any correct prediction.

What the classifier gets wrong. Of its 11 test errors, 8 had a top score below 0.5, and in 6 the correct identity was its second guess, so most mistakes are uncertain rather than confident. Nine of the 11 errors involve the same five identities (celeb_2970, 4422, 4428, 7007 and 10002), all women with long hair; for example celeb_4428 was twice predicted as celeb_2970. The other two errors are our own celeb_1212, once named as another dark-haired man (celeb_2336) and once as celeb_5695, both times with the correct name as second guess. With about 17 training photos per person the model leans on hair, pose and lighting as much as on facial features.

Limitations. (1) *Small data:* 17 training photos per identity; the validation/test splits have only 50 photos and 12 grids, so differences of a few faces between settings are within noise. (2) *Synthetic images:* faces are clean crops on plain backgrounds at a narrow range of sizes; very small faces (2/18 correct when shrunk to 45%) and real group photos with overlapping faces, different lighting and occlusion would be much harder. (3) *Closed set:* the system always picks one of the 13 identities. A stranger will be given a celebrity's name; the "?" marker for low scores only partly mitigates this, and we did not evaluate an

"unknown person" option. (4) *One split, one seed:* results come from a single data split; repeated splits would give error bars. (5) The larger YOLOv8s scored higher on test (mAP@0.5 0.936) but lower on validation, so we did not select it; it is the first candidate for future work, along with training on smaller and overlapping faces and adding an open-set "unknown" class.

8. Ethical considerations

Face recognition is one of the most ethically sensitive applications of computer vision, and even a small classroom system like ours shows the main risks clearly.

Bias and unequal accuracy. Our system is far more accurate for some people than others: on test photos, accuracy ranges from 100% for seven identities to 25% for one. The errors are not random: 9 of the 11 classifier mistakes involve the same five women with similar hair. In our small set this reflects visual similarity and too few photos rather than a measured demographic effect, but it is exactly how demographic bias arises in real systems: when some groups are under-represented, or represented by less varied photos, the model learns weaker features for them and fails more often. Large independent evaluations have found the same pattern in commercial systems: NIST's study of demographic effects in face recognition measured error rates that differ substantially across sex, age and race, and the Gender Shades audit found commercial face-analysis systems misclassified darker-skinned women far more often than lighter-skinned men. CelebA itself is not demographically balanced: it consists of celebrities photographed at public events, skewed toward particular ages, ethnicities and styles of presentation. A model trained on it should not be assumed to work equally well for everyone. A responsible version of this system would report accuracy per demographic group, collect more balanced data, and refuse to give a name when its confidence is low, rather than reporting only one average number. Our per-identity tables are a small step in that direction.

Privacy and consent. The people in CelebA did not consent to having their faces used to train recognition models. The photos were collected from the internet, and although the dataset is licensed for non-commercial research, being a public figure does not imply agreement to be automatically identified. A face is a permanent biometric identifier: unlike a password it cannot be changed if it leaks. Many jurisdictions now treat face templates as sensitive data that needs explicit consent (for example the GDPR in the EU and Illinois' BIPA). For a real deployment, consent, a clear purpose, limits on how long images and face embeddings are stored, and a way for people to opt out would be required. In this project we used the data only for coursework, did not collect any new images of people, and did not build a database of embeddings.

Misuse and dual use. The same pipeline that names celebrities in a grid could, with a different gallery of faces, identify private individuals in street, protest or classroom photos: tracking people, stalking, or targeting activists. Even with good intentions, misidentification causes real harm. There are documented cases of wrongful arrests based on incorrect face matches, and our system illustrates why: being closed-set, it always outputs *someone's* name, often with high confidence, even for a person it has never seen. Mitigations include restricting use to consenting, enrolled individuals; showing uncertainty (our "?" marker) instead of a single confident answer; keeping a human in the loop for any decision that affects a person; logging and auditing use; and not releasing

models trained to recognize specific people. We publish only code and models for 13 public figures from a research dataset, and the system is not intended or suitable for identifying anyone outside that set.

What we take from this. Accuracy alone is not the right measure of a face-recognition system. Who it fails for, whether people agreed to be in it, and what happens when it is wrong matter as much as the average score. Small data made these questions concrete for us: with 17 photos per person our model was much better at some people than others.

9. Conclusion

We built an end-to-end system that finds and names 13 celebrities in multi-face images. A synthetic grid dataset with a strict photo-level split made evaluation honest; a fine-tuned YOLOv8n solved localization (IoU 0.99); and combining it with a fine-tuned ResNet18 classifier raised the share of correctly identified test faces from 77.8% to 82.4% while removing all duplicate boxes. The remaining errors are confusions between similar-looking identities, driven by having only about 17 photos per person, and they are a reminder that face recognition must be judged by who it fails for as well as how often.

Reproducibility. `pip install -r requirements.txt`, then `python src/run_milestone3.py` (detector, GPU recommended) and `python src/run_milestone4.py` (classifier, pipeline evaluation and gallery; runs on CPU). All splits are deterministic (seed 42), and trained checkpoints are committed in `models/`.

Team contributions. *Abdellah Faleh:* Milestone 1 data pipeline and models; Milestone 2 dataset pipeline; Milestone 3 scaled dataset, training sweep, evaluation and report; Milestone 4 integrated pipeline, classifier retraining, gallery and final report. *Jin-Woo Hong:* repository setup and maintenance; Milestone 2 synthetic face-frame pipeline; Milestone 3 baseline on the original dataset, evaluation scripts and live-demo notebook. *Samuel Tong:* evaluation and review across milestones, Milestone 3 evaluation review, rubric completeness. *Tristan Lyons:* planning, code and results review, submission checklist, Canvas submissions and coordination with the TA. All members contribute to the final presentation.

References.

- Z. Liu, P. Luo, X. Wang, X. Tang. *Deep Learning Face Attributes in the Wild* (CelebA). ICCV 2015.
- K. He, X. Zhang, S. Ren, J. Sun. *Deep Residual Learning for Image Recognition.* CVPR 2016.
- G. Jocher, A. Chaurasia, J. Qiu. *Ultralytics YOLOv8*, 2023. https://github.com/ultralytics/ultralytics
- P. Grother, M. Ngan, K. Hanaoka. *Face Recognition Vendor Test Part 3: Demographic Effects.* NIST IR 8280, 2019.
- J. Buolamwini, T. Gebru. *Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification.* FAT* 2018.